Mawlana Bhashani Science and Technology University

Department of Information and Communication Technology

PROJECT PROPOSAL

AI Study Planner

A Student-Centered Academic Task and Study Planning Application

Submitted in partial fulfilment of the course requirements for

Software Engineering and Project Management (ICT-31007)

Submitted by	Mehejabin Jannat Mourin
Student ID	IT-24062
Submitted to	Prof. Dr. Ziaur Rahman
Institution	Mawlana Bhashani Science and Technology University
Proposal date	27 October 2026

Contents

Abstract	2
1. Introduction and Background	3
2. Problem Statement	4
3. Aim and Objectives	5
4. Scope of the Project	6
5. Proposed System	7
6. Stakeholders and User Roles	8
7. Functional Requirements	9
8. Non-Functional Requirements	10
9. System Design and Architecture	11
10. Methodology and Development Process	12
11. Technology Stack	13
12. Testing and Evaluation Plan	14
13. Project Schedule	15
14. Risk Analysis and Mitigation	16
15. Ethical, Privacy, and Security Considerations	17
16. Expected Outcomes	18
17. Future Enhancements	19
18. Conclusion	20
References	21

Page references are indicative and may shift slightly with pagination.

Abstract

Students regularly manage lectures, assignments, quizzes, examinations, projects, and personal commitments. When these responsibilities are tracked across notebooks, messages, and separate applications, students may miss deadlines or struggle to decide what to study next. The proposed AI Study Planner is a student-focused planning application intended to bring subjects, tasks, deadlines, daily study plans, and progress information into one place. The planned mobile interface uses Flutter and Dart, with Firebase Authentication and Cloud Firestore as the intended identity and data services. A Node.js and Express.js backend is planned for task-related API operations and future scheduling services. The initial recommendation logic will be transparent and rule-based, using task priority, due date, and estimated duration. It will not be represented as a trained AI model until an AI component is implemented and evaluated. The project will be developed iteratively, with requirements analysis, system design, prototype development, implementation, testing, and documentation. The expected result is a usable prototype that demonstrates task management, suggested study ordering, and progress tracking, with clear opportunities for later personalization and AI-based recommendations.

1. Introduction and Background

University students must balance several courses and deadlines within a limited amount of time. A study plan can make academic responsibilities more visible and help students distribute effort across a semester. However, creating a useful plan manually requires regular updates and a clear view of task urgency and progress. The AI Study Planner is proposed as a centralized digital application for recording study tasks and turning them into a practical daily plan. The project is designed as a modular student-support tool that can begin with basic scheduling rules and later incorporate validated personalization features.

2. Problem Statement

Students may record academic tasks in different places, forget deadlines, underestimate the time required for preparation, or repeatedly spend time deciding which task to start. Generic calendar applications can store events, but they may not provide a focused view of subjects, study tasks, completion status, and recommended study order in a single student-oriented workflow. The project will address this gap by providing a simple interface for organizing academic work and prioritizing tasks based on clear, explainable rules.

3. Aim and Objectives

3.1 Aim

To design and develop a student-oriented study planning prototype that helps users organize academic tasks, prioritize upcoming work, and monitor study progress.

3.2 Specific Objectives

- Allow users to create and manage subjects and study tasks.
- Record task titles, descriptions, due dates, estimated duration, priority, and completion status.
- Generate a suggested daily task order using transparent priority and deadline rules.
- Display completed and pending tasks and summarize progress.
- Design a modular architecture that can support authentication, cloud storage, reminders, and future recommendation features.
- Evaluate usability and functional correctness through test cases and user feedback.

4. Scope of the Project

4.1 Included in the Initial Scope

- A mobile-oriented user interface built with Flutter and Dart.
- Subject and task creation, viewing, editing, and completion workflows.

- Task prioritization using due dates, urgency, priority level, and estimated duration.
- A dashboard for pending tasks and completion progress.
- A prototype and supporting software-engineering diagrams.
- A Node.js/Express API structure and planned Firebase data integration.

4.2 Outside the Initial Scope

- A production-ready, trained AI model.
- Guaranteed prediction of grades or academic performance.
- Automated access to a university learning-management system.
- Production deployment, paid cloud services, and real-time notifications unless configuration and testing are completed.
- Replacement of a student's own judgement or instructor guidance.

5. Proposed System

The proposed system will provide a central dashboard where a student can view today's study tasks, add new tasks, mark work as complete, and review progress. Task suggestions will be generated by a rule-based scheduling component. For example, a task with a near deadline and high priority can be placed before a low-priority task due later. The user should be able to understand why a task is recommended and should retain control over the final schedule.

The planned architecture has three main layers:

1. Frontend: Flutter and Dart provide the mobile interface and interaction flow.
2. Backend: Node.js and Express.js expose task-related API routes and provide a place for future scheduling logic.
3. Data services: Firebase Authentication and Cloud Firestore are the intended services for user identity and task storage.

The current repository contains a starter Flutter interface, an Express server and task route structure, a prototype built with HTML/CSS/JavaScript, and source files for system diagrams. Firebase configuration and production credentials must be supplied separately and tested before the system can be described as fully integrated.

6. Stakeholders and User Roles

Stakeholder	Role / Interest
Student / end user	Creates subjects and tasks, follows suggested plans, and reviews progress
Project developer	Gathers requirements, designs, implements, tests, and documents the system
Course instructor	Reviews project scope, software-engineering artefacts, and demonstration
Future evaluator	Assesses usability, correctness, and potential for further development

The initial version is intended for individual students. Administrative dashboards and institutional-level management are not part of the initial scope.

7. Functional Requirements

ID	Requirement	Priority
FR-01	The system shall display a dashboard of current study tasks.	Must
FR-02	The user shall be able to add a task with title, subject, due date, and estimated duration.	Must
FR-03	The user shall be able to edit or delete a task.	Must
FR-04	The user shall be able to mark a task as complete or pending.	Must
FR-05	The system shall assign or store a priority level for each task.	Must
FR-06	The system shall generate a suggested task order using deadline and priority rules.	Should
FR-07	The dashboard shall show a summary of completed and pending work.	Should
FR-08	The system should support user sign-in when Firebase Authentication is configured.	Should
FR-09	The system should persist user tasks to the configured database.	Should
FR-10	The system may support reminders and personalized recommendations in a future release.	Could

8. Non-Functional Requirements

- Usability: Common task actions should be easy to locate and require minimal steps.
- Performance: Dashboard and task operations should respond promptly under normal prototype conditions.
- Reliability: Invalid or incomplete task data should be handled without crashing the application.
- Security: User-specific data should be protected by authentication and correctly configured database rules before deployment.
- Maintainability: Frontend, backend, data model, and diagrams should remain organized in separate modules.
- Portability: The Flutter interface should be designed for Android first, with other supported platforms considered later.
- Transparency: Suggested study ordering should be explainable and editable by the student.

9. System Design and Architecture

9.1 High-Level Workflow

1. The student opens the application and views the dashboard.
2. The student creates or updates a subject or task.
3. The application validates the task information.
4. The task is stored locally during prototype use or sent to the backend and configured data service.
5. The planner evaluates task priority, due date, and estimated duration.
6. The dashboard presents a suggested study order and updated completion progress.

9.2 Data Model

A task record is expected to contain an identifier, user identifier, subject, title, optional description, due date, estimated duration, priority, status, and creation/update timestamps. A subject record may contain an identifier, user identifier, subject name, and optional color or note. The final schema may be refined during implementation.

9.3 Main Modules

- Authentication and user profile

- Subject management
- Study task management
- Daily plan and priority ordering
- Progress dashboard
- Settings and future reminders

9.4 Design Artefacts

The repository includes or plans to include a use-case diagram, class diagram, object diagram, activity diagram, context-level data-flow diagram, level-1 data-flow diagram, and entity-relationship diagram. Diagram source files will be maintained alongside rendered images so that designs can be revised as requirements change.

10. Methodology and Development Process

An iterative development process will be used. Each iteration will produce a small, reviewable part of the system and will be checked against the requirements.

1. Requirement analysis: Identify core user needs, functional requirements, constraints, and project boundaries.
2. System design: Prepare use cases, data model, architecture, interface sketches, and workflow diagrams.
3. Prototype development: Build the main user journey and test layout and navigation.
4. Implementation: Develop Flutter screens, backend routes, and database integration in stages.
5. Testing: Perform unit-level checks, integration checks, validation tests, and usability review.
6. Documentation: Update the README, user instructions, test evidence, diagrams, and final report.
7. Review and refinement: Resolve issues identified during testing and record remaining limitations.

11. Technology Stack

Component	Planned Technology	Purpose
Mobile frontend	Flutter, Dart	Cross-platform user interface
Backend API	Node.js, Express.js	Task endpoints and future scheduling services
Authentication	Firebase Authentication	User sign-in when configured
Database	Cloud Firestore	Persistent user and task records when configured
Web prototype	HTML, CSS, JavaScript	Demonstration of interface concepts
Design documentation	PlantUML, Mermaid, PNG	Software-engineering diagrams
Version control	Git and GitHub	Source management and project history

12. Testing and Evaluation Plan

Testing will focus on functional correctness, input validation, consistency of task status, priority ordering, and ease of use. The developer will create test cases for valid task creation, missing required fields, editing and deleting tasks, marking tasks complete, handling overdue tasks, and ordering tasks with different priorities and due dates. Backend endpoints will be tested independently before integration. Database and authentication tests will only be marked complete after actual Firebase configuration is supplied.

Evaluation will record whether each requirement passes, fails, or remains pending. Informal usability feedback may be collected from a small number of student users, with consent and without collecting unnecessary personal information. The project will not claim statistically validated learning improvements unless a suitable study is designed and conducted.

13. Project Schedule

Phase	Planned Activities	Deliverable
1	Requirements and background review	Requirements list
2	Use cases, architecture, data model	Design diagrams
3	Interface and navigation prototype	Clickable / web prototype
4	Flutter screens and task operations	Frontend implementation
5	API and Firebase configuration	Integrated task workflow
6	Functional and integration testing	Test results
7	Corrections and documentation	Final repository and report
8	Demonstration and presentation	Project demonstration

The exact duration of each phase will be adjusted according to course deadlines, implementation progress, and access to required services.

14. Risk Analysis and Mitigation

Risk	Impact	Mitigation
Limited development time	Some features may remain incomplete	Prioritize core task workflows and maintain a clear minimum viable product
Firebase setup errors	Cloud persistence or sign-in may not work	Configure credentials securely and test each service independently
API integration issues	Frontend and backend may behave inconsistently	Define request/response formats and test endpoints separately
Recommendation quality is limited	Suggested plans may not fit every student's needs	Use explainable rules first and allow manual edits
Scope expansion	Delivery may be delayed	Keep advanced AI, notifications, and analytics as future work
Privacy or access-rule mistakes	User data could be exposed	Apply least-privilege rules and avoid committing credentials to GitHub

15. Ethical, Privacy, and Security Considerations

The system should collect only information required for study planning. User-specific records should be accessible only to their authorized owner. Credentials and private configuration files must not be committed to a public repository. Any feedback collected during evaluation should be voluntary and should not include unnecessary personal or academic records. Recommendations should be presented as assistance rather than authoritative decisions about a student's ability or future performance.

16. Expected Outcomes

- A structured project repository with source code and documentation.
- A mobile-oriented study planner interface.
- Core task and subject management workflows.
- A transparent, rule-based daily task ordering feature.
- A progress summary for completed and pending tasks.
- System design diagrams and a documented test plan.
- A clear foundation for later Firebase integration, reminders, and AI-based personalization.

17. Future Enhancements

Future versions may include calendar synchronization, configurable study reminders, weekly analytics, adaptive planning based on user feedback, and an AI recommendation service. Any AI component should be evaluated against a baseline rule-based planner and should explain its recommendations in an understandable way. These features are proposals, not claims about functionality already completed.

18. Conclusion

The AI Study Planner proposes a practical way to organize academic responsibilities in a single student-oriented application. Its initial focus is on dependable task management, transparent prioritization, and progress visibility. A modular Flutter, Node.js, and Firebase-oriented design can support incremental development while keeping advanced features outside the initial scope until they are implemented and tested. The project will be assessed through functional testing, documentation, and demonstration of the working prototype.

References

1 Flutter. (n.d.). *Flutter documentation*. https://docs.flutter.dev/

2 Dart. (n.d.). *Dart language documentation*. https://dart.dev/guides

3 Firebase. (n.d.). *Firebase documentation*. https://firebase.google.com/docs

4 Express.js. (n.d.). *Express web framework documentation*. https://expressjs.com/

5 Node.js. (n.d.). *Node.js documentation*. https://nodejs.org/docs/

6 Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1